Software Design Document — Hierarchical VPT for BanglaMedVQA

Software Design Document — Hierarchical VPT for BanglaMedVQA

Repository: `med-vqa-LoRA-vpt-test` **Status of this document:** updated by reviewer pass on 2026-04-27. **Previous baseline:** `AhmedRafid023/med-vqa-LoRA` — LoRA-only fine-tuning of VLMs via LLaMA Factory. **Reference plan:** hierarchical_vpt_protocol.md

This document describes (a) the entire codebase as it exists after the reviewer pass, (b) what was changed relative to the LoRA-only baseline, (c) every fix applied during review, and (d) how to train and evaluate on Lightning.ai's cloud GPUs.

---

1. Goal

The paper that this codebase reproduces fine-tunes Qwen2.5-VL-7B and Med-Gemma-4B on BanglaMedVQA with LoRA on the language model. After fine-tuning, two question categories — **Condition (18.5%)** and **Position (31.3%)** — remain catastrophically weak because LoRA only adapts the language model: the visual grounding failure lives in the frozen ViT.

This work adds Hierarchical Visual Prompt Tuning (VPT) on top of the LoRA baseline:

- Two pools of learnable prompt tokens injected into every ViT transformer block.
  - **Gen pool** — used for *Modality* and *Organ* questions (broad, image-level cues).
  - **Spec pool** — used for *Abnormality*, *Condition*, *Position* (fine-grained, region-level cues).
- A **Category Router** (small MLP on a frozen multilingual sentence embedding of the question) outputs a routing weight `alpha ∈ [0, 1]` that softly blends the two pools per sample.
- LoRA on the LM is kept (identical hyper-parameters to the paper) and trained jointly in a two-phase schedule.

Five ablations isolate each component (A1 → A5; see §6).

---

2. Repository layout

```
med-vqa-LoRA-vpt-test/
├── src/
│   └── vpt/
│       ├── __init__.py              # public exports
│       ├── prompt_learner.py        # Deep VPT pools + ViT forward hooks
│       ├── category_router.py       # multilingual MLP router
│       └── vpt_model.py             # combined VLM + VPT + router wrapper
├── configs/
│   ├── qwen2.5-vl-7b.yaml           # LLaMA-Factory baseline (B0) — UNCHANGED
│   ├── qwen2.5-vl-7b-test.yaml      # baseline test config — UNCHANGED
│   ├── med-gemma-4b.yaml            # baseline (Med-Gemma) — UNCHANGED
│   ├── med-gemma-4b-test.yaml       # baseline test (Med-Gemma) — UNCHANGED
│   ├── gemma-3.yaml / gemma-3-test.yaml
│   └── vpt/
│       ├── A1_shared_no_router.yaml # equal-blend, no router, no LoRA
│       ├── A2_shared_with_router.yaml
│       ├── A3_split_with_router.yaml
│       ├── A4_split_router_lora.yaml   # PROPOSED METHOD
│       └── A5_oracle_router.yaml       # oracle upper bound
├── data/
│   ├── chest_x-ray/{train,test}/    # CSV + dataset.json + images
│   ├── medicat/{train,test}/
│   └── dataset_info.json            # LLaMA-Factory dataset registry
├── prepare_data.py                  # CSV → ShareGPT JSON (UNCHANGED)
├── train_model.py                   # baseline B0 trainer / test inference
├── train_vpt.py                     # NEW — VPT trainer
├── infer_vpt.py                     # NEW — VPT inference → predictions CSV
├── evaluate.py                      # NEW — per-category Acc / BERTScore / LAVE
├── requirements.txt                 # baseline deps
├── requirements_vpt.txt             # additional VPT deps
├── hierarchical_vpt_protocol.md     # research/design plan
└── SOFTWARE_DESIGN.md               # THIS DOCUMENT
```

What is **new** vs. the LoRA-only baseline (`AhmedRafid023/med-vqa-LoRA`):

Path	Status
`src/vpt/*`	**New** — entire VPT package
`train_vpt.py`	**New** — VPT-specific training loop with two-phase schedule
`infer_vpt.py`	**New** — VPT inference (loads VPT checkpoints + LoRA adapter)
`evaluate.py`	**New** — Acc / BERTScore / LAVE per category (matches paper Tables 4 & 5)
`configs/vpt/A1..A5.yaml`	**New** — ablation presets

Path	Status
`requirements_vpt.txt`	**New** — `sentence-transformers`, `bert-score`, `openai`
`train_model.py`	**Modified** — `run_real_inference()` now writes the `category` column required by `evaluate.py`
`README.md`	Modified — high-level project description
All other files	unchanged from baseline

---

3. Component design — detailed

This section walks through every class and every significant method in the codebase. For each piece of code it explains the theoretical concept that motivates it, the concrete implementation decision, and the consequences of getting it wrong.

Background concepts you need first

What is a Vision-Language Model (VLM)? A VLM like Qwen2.5-VL-7B is a two-part system. The first part is a Vision Transformer (ViT): it slices an image into small patches (e.g. 14×14 pixels each), turns each patch into a vector via a learned linear projection, prepends a special `[CLS]` token, and runs the resulting *sequence of patch tokens* through many transformer blocks. Each block performs self-attention (every token attends to every other token), so spatial relationships between patches are learned across layers. The output of the final ViT block is a sequence of visual feature vectors.

The second part is a Large Language Model (LLM) like Qwen2-7B. The visual features are projected (by a "visual merger" or "connector" module) into the same embedding space as text tokens, concatenated with the tokenised question text, and then forwarded through the LLM to produce an answer autoregressively — predicting one token at a time.

Why does LoRA alone fail for Condition and Position questions? LoRA (Low-Rank Adaptation) adds small trainable matrices to the attention and feed-forward layers *inside the LLM*. It teaches the language part of the model to produce better Bangla answers. But the ViT — which is responsible for extracting visual features — is kept frozen. For coarse questions like "what imaging modality is this?" the language model can answer from global visual statistics. For fine-grained questions like "where is the lesion?" or "is the opacity in the upper-right lobe?" the answer depends on specific spatial regions of the image that the frozen ViT must correctly encode. LoRA cannot fix this because it never sees the image; it only sees the projected visual tokens after they leave the ViT.

What is Visual Prompt Tuning (VPT)? VPT (Jia et al., 2022) inserts a small set of extra learnable tokens — called *prompt tokens* — into the token sequence that the ViT processes. Because these extra tokens attend to (and are attended by) the real patch tokens at every layer, they effectively bias the ViT's attention patterns. The key insight is that you can significantly change *what a frozen ViT notices in an image* by tuning only these small prompt vectors, without touching any of the billions of ViT weights.

There are two flavors: - **Shallow VPT**: prompts are added only at the input to the first ViT block. They influence only the first layer's attention and then get mixed into the patch representations before the second layer. - **Deep VPT**: fresh prompt tokens are added at the input of *every* ViT block. Each layer gets its own dedicated set of learnable prompts, giving maximum expressiveness. This codebase uses Deep VPT.

What makes this work *hierarchical*? The novelty here is that instead of one shared set of VPT tokens, there are two *pools* with semantically different purposes, and a learned router picks which pool (or what blend of pools) to use based on the question. Modality/Organ questions are visually global (the model just needs to recognize broad structure), so a "general" prompt pool is appropriate. Condition/Position questions are visually local and specific, so a "specialist" pool that has been optimized for fine-grained grounding is more appropriate. The hierarchical design lets the ViT adapt differently depending on what kind of question is being asked — something a single shared pool cannot do as well.

3.1 `src/vpt/prompt_learner.py` — `VPTPromptLearner`

Class-level purpose. This module owns the learnable tensors (the two prompt pools) and the hook machinery that injects them into the frozen ViT at runtime. It is an `nn.Module` solely so that `gen_prompts` and `spec_prompts` are registered as parameters (and therefore visible to the optimizer and checkpoint serialization). Its actual computation happens entirely inside PyTorch forward hooks — the `forward()` method is a no-op.

`__init__(self, vit_model, num_gen_tokens, num_spec_tokens, prompt_dim, num_layers)`

```
self.num_gen_tokens  = num_gen_tokens   # e.g. 16
self.num_spec_tokens = num_spec_tokens  # e.g. 24
self.prompt_dim      = prompt_dim       # e.g. 1024 (ViT hidden size)
self.num_prompt_tokens = max(num_gen_tokens, num_spec_tokens)  # 24
```

`num_prompt_tokens` is a derived constant — the *maximum* of the two pool sizes. This is the number of tokens that will be prepended to the hidden state at every layer. Why always use the max? Because Deep VPT requires a constant sequence-length increase across all layers.

If layer 0 injects 24 tokens and layer 1 expects to strip 24 and then inject 16, the math is clean only when the “strip” count equals the “inject” count. By zero-padding the smaller pool (Gen, 16 tokens) up to 24 before blending, the injected block is always exactly 24 tokens wide regardless of alpha.

```
self.gen_prompts = nn.Parameter(torch.zeros(num_layers, num_gen_tokens,
        prompt_dim))
self.spec_prompts = nn.Parameter(torch.zeros(num_layers, num_spec_tokens,
        prompt_dim))
nn.init.trunc_normal_(self.gen_prompts,  std=0.02)
nn.init.trunc_normal_(self.spec_prompts, std=0.02)
```

Shape: `[num_layers, num_tokens, prompt_dim]`. The first dimension indexes the ViT layer, the second indexes the prompt token within that layer, the third is the hidden dimension. This is a 3-D tensor of parameters, not a 2-D weight matrix — `nn.Parameter` accepts tensors of any shape.

Why not start from zero? Zero initialization gives zero gradient on the first backward pass because the blend formula `alpha * g + (1-alpha) * s` produces a zero blended tensor, and zero × anything = zero gradient. Truncated normal with `std=0.02` (same as BERT/ViT embedding initialization) gives a nonzero but small starting point.

```
self._hooks: list = []
self._current_alpha = None
```

`_hooks` stores `RemovableHook` objects returned by `register_forward_pre_hook` and `register_forward_hook`. They must be stored so they can be removed when `_remove_hooks()` is called (important during re-registration or when the model is reused across ablations). `_current_alpha` is the per-batch routing weight — it is set by `set_routing_weight()` before every model forward call and read inside the hooks during that forward pass.

`_count_vit_layers(self, vit_model) → int`

```
if hasattr(vit_model, 'visual') and hasattr(vit_model.visual, 'blocks'):
    return len(vit_model.visual.blocks)   # Qwen2.5-VL
if hasattr(vit_model, 'vision_tower'):
    return len(vit_model.vision_tower.vision_model.encoder.layers)  # MedGemma
raise ValueError(...)
```

This is model-architecture introspection. Different VLMs bury their ViT at different attribute paths. The module checks for the two known patterns and raises a helpful error for anything else.

- **Qwen2.5-VL**: the model class is `Qwen2_5_VLForConditionalGeneration`. It has a `visual` attribute which is a `Qwen2_5_VisionTransformerPretrainedModel`. Inside that, the actual transformer blocks are a list called `blocks`. The number of blocks in Qwen2.5-VL-7B is 32.
- **MedGemma / SigLIP**: `vision_tower` → `vision_model` → `encoder` → `layers`. This is the standard CLIP/SigLIP naming pattern used by PaLI and Gemma-based vision models. MedGemma-4B has 27 layers in its SigLIP encoder.

`_get_vit_blocks(self, vit_model) → list`

```
if hasattr(vit_model, 'visual') and hasattr(vit_model.visual, 'blocks'):
    return list(vit_model.visual.blocks)
return list(vit_model.vision_tower.vision_model.encoder.layers)
```

Returns the actual list of block modules. Hooks need to be registered on individual block objects (`nn.Module` instances), not on the container. `list(...)` materializes the `nn.ModuleList` into a plain Python list so we can safely index it with `enumerate`.

`register_hooks(self, vit_model)`

```
self._remove_hooks()
blocks = self._get_vit_blocks(vit_model)
for i, block in enumerate(blocks):
    is_last = (i == len(blocks) - 1)
    h = block.register_forward_pre_hook(
        lambda module, args, li=i: self._inject_tokens(args, li)
    )
    self._hooks.append(h)
    if is_last:
        h = block.register_forward_hook(
            lambda module, _args, output: self._strip_tokens(output)
        )
        self._hooks.append(h)
```

This is the core wiring step. `register_forward_pre_hook(fn)` on a module causes `fn(module, args)` to be called just before that module's `forward()` runs, and its return value replaces `args`. So by returning a modified `args[0]` (the hidden state), we can change what the block actually processes without touching the block's code.

The `li=i` capture in the lambda is critical. Python closures capture variables by reference, not by value. Without `li=i`, every lambda would close over the same loop variable `i` and all hooks would use the final value of `i` (the index of the last block) — a classic Python bug. The `li=i` in the default argument captures the current value of `i` at loop iteration time.

`_remove_hooks()` is called first so that re-calling `register_hooks()` (e.g. after loading a checkpoint) does not double-up hooks.

The post-hook on the last block is separate from the pre-hooks. `register_forward_hook(fn)` calls `fn(module, input, output)` after the block's `forward()` has finished, letting us modify the output. This is where we strip the prompt tokens before the visual merger sees the sequence.

`_inject_tokens(self, args, layer_idx) → tuple`

This is the most important method in the whole codebase. Let's walk through it line by line.

```
hidden = args[0]    # shape: [B, seq_len, D]
B = hidden.shape[0]

if layer_idx > 0:
    hidden = hidden[:, self.num_prompt_tokens:, :]
```

`args[0]` is the hidden state tensor entering this ViT block. Its shape at the start of every block is `[batch_size, sequence_length, hidden_dim]`. At layer 0, `sequence_length` is the original ViT sequence: `1 + num_patches` (1 for CLS token, then one token per image patch). At layers 1 and beyond, the sequence is `num_prompt_tokens + 1 + num_patches` because the previous layer's pre-hook injected `num_prompt_tokens` tokens and they traveled through the block's self-attention as part of the sequence. We must strip those stale tokens before injecting fresh ones, otherwise the sequence grows unboundedly.

The slice `[:, self.num_prompt_tokens:, :]` discards the first `num_prompt_tokens` positions (the old prompt tokens), keeping everything from position `num_prompt_tokens` onward (the real patch tokens).

```
alpha = self._current_alpha  # shape: [B, 1, 1] or None, float32

gen_tok  = self.gen_prompts[layer_idx].unsqueeze(0).expand(B, -1, -1)   # [B, G,
        D]
spec_tok = self.spec_prompts[layer_idx].unsqueeze(0).expand(B, -1, -1)  # [B, S,
        D]
```

Index into the parameter pool to get this layer's tokens. `self.gen_prompts[layer_idx]` has shape `[num_gen_tokens, prompt_dim]`. `.unsqueeze(0)` makes it `[1, G, D]`, and `.expand(B, -1, -1)` broadcasts it to `[B, G, D]` without copying memory. Each sample in

the batch sees the same prompt tokens (they are not sample-specific), but alpha will differentiate them.

```
N = self.num_prompt_tokens  # 24
g = torch.zeros(B, N, self.prompt_dim, device=hidden.device, dtype=hidden.dtype)
s = torch.zeros(B, N, self.prompt_dim, device=hidden.device, dtype=hidden.dtype)
g[:, :self.num_gen_tokens,  :] = gen_tok   # fill first G slots
s[:, :self.num_spec_tokens, :] = spec_tok  # fill first S slots
```

Pad both pools to width `N = max(G, S) = 24`. Gen pool (16 tokens) goes into the first 16 positions of `g`; the last 8 positions remain zero. Spec pool (24 tokens) fills all of `s`. Why pad to max rather than concatenate? The blend formula `alpha * g + (1-alpha) * s` requires both tensors to have the same shape. Padding with zeros means the zero-padded tail positions contribute zero to the blend and produce zero-gradient for those positions — a natural no-op. **Crucially, both `g` and `s` are created with `dtype=hidden.dtype` (bfloat16), not float32.** This is where the dtype fix lives.

```
if alpha is None:
    blended = 0.5 * g + 0.5 * s
else:
    a = alpha.to(dtype=hidden.dtype) if isinstance(alpha, torch.Tensor) else
        alpha
    blended = a * g + (1.0 - a) * s   # [B, N, D]
```

For A1 (no router, `disable_router=True`): alpha is `None`, so both pools contribute equally. For all other ablations, alpha is a tensor of shape `[B, 1, 1]` (reshaped in `set_routing_weight`). The `[B, 1, 1]` shape broadcasts correctly against `[B, N, D]`: each sample in the batch gets its own scalar alpha applied uniformly across all N tokens and all D dimensions. The `.to(dtype=hidden.dtype)` cast is the fix for the dtype bug — the router MLP operates in float32, but we must cast to bfloat16 to match the ViT before the multiplication. The **original** `alpha` tensor (float32, carrying the autograd computation graph) is still stored in `self._current_alpha` and returned to the training loop for the loss computation.

```
hidden_with_prompts = torch.cat([blended, hidden], dim=1)  # [B, N+seq, D]
return (hidden_with_prompts,) + args[1:]
```

Prepend the prompt tokens at the *front* of the sequence. The choice to prepend (rather than append) follows the original VPT paper convention and is practically important: many ViT implementations apply positional embeddings only to the original sequence positions, so appended tokens would receive no positional bias. Prepended tokens also attend to all patch tokens starting from layer 0, giving maximum influence. The hook returns a tuple

`(modified_hidden, *rest_of_args)` which replaces the original `args` fed to the block's `forward()`.

`_strip_tokens(self, output) → tensor or tuple`

```
if isinstance(output, tuple):
    hidden = output[0][:, self.num_prompt_tokens:, :]
    return (hidden,) + output[1:]
return output[:, self.num_prompt_tokens:, :]
```

After the last ViT block has processed the `[B, N+seq, D]` sequence, we need to return it to the shape `[B, seq, D]` that the visual merger expects. The visual merger was designed to receive the original `seq`-length ViT output and project it to LLM token space. If we pass it `N+seq` tokens, the projection dimension changes and the LLM sees more tokens than it expects from its positional encodings and context.

The `isinstance(output, tuple)` branch handles the case where a ViT block returns a tuple `(hidden_state, attention_weights)` rather than just the tensor — SigLIP blocks do this. In that case we strip only the first element and reconstruct the tuple.

`set_routing_weight(self, alpha)`

```
if isinstance(alpha, torch.Tensor):
    self._current_alpha = alpha.view(-1, 1, 1)
else:
    self._current_alpha = alpha  # None or scalar
```

Called from `HierarchicalVPTModel.forward()` with the router's output before calling `self.model(...)`. It stores `alpha` on the instance so the hooks (which are closures that reference `self`) can pick it up during the upcoming ViT forward pass. The `.view(-1, 1, 1)` reshapes `[B]` to `[B, 1, 1]` so that it broadcasts cleanly against `[B, N, D]` tensors in `_inject_tokens`. `None` passes through as-is and triggers the 0.5-blend fallback.

Why not pass alpha as an argument to the hook directly? PyTorch's hook signature is fixed: pre-hooks receive `(module, args)` and return new args. There is no channel to pass extra state into a hook at call time. The standard solution is to store shared state on the parent object — which is exactly what `_current_alpha` is.

`_remove_hooks(self)`

```
for h in self._hooks:
    h.remove()
self._hooks = []
```

Each `h` is a `RemovableHook`. Calling `.remove()` detaches the hook from the module it was registered on. This is essential for two scenarios: (1) calling `register_hooks()` again (e.g. after loading a new checkpoint) — you must remove the old hooks first or you get double injection; (2) when the model is garbage-collected — hooks hold references to the module, potentially preventing GC.

`forward(self)`

```
def forward(self):
    pass
```

This method is required for `nn.Module` compliance but does nothing. All the computation happens via the side-effectful hooks. There is no direct data path through `VPTPromptLearner.forward()`.

3.2 `src/vpt/category_router.py` — `CategoryRouter`

Theoretical background. The router's job is to look at a Bangla question string and decide whether it is a "general" question (Modality/Organ) or a "specific" question (Abnormality/Condition/Position). This is a text classification problem, but with an unusual constraint: the dataset is in Bangla and the question vocabulary is limited (most questions are slight rephrases of 5 templates). Rather than training a Bangla-specific classifier from scratch, we use a pre-trained multilingual sentence encoder that has already learned to represent semantically similar sentences as nearby vectors — even across languages — and add a tiny trainable MLP on top.

Why a sentence encoder rather than the VLM's own text encoder? Using the VLM tokenizer + language model backbone to embed questions would require a separate forward pass through the LLM on every training step, doubling compute. A lightweight frozen sentence encoder (~110M parameters) gives good sentence representations in ~20 ms per batch with no additional gradient computation through it.

Module-level constants

```
CATEGORY_TO_LABEL = {
    "modality":    0,  # Gen
    "organ":       0,  # Gen
    "abnormality": 1,  # Spec
    "condition":   1,  # Spec
    "position":    1,  # Spec
}
_ENCODER_MODEL = "paraphrase-multilingual-mpnet-base-v2"
```

The binary label scheme: 0 = “this question needs the Gen pool,” 1 = “this question needs the Spec pool.” The choice to map Modality and Organ to Gen (0) and the three harder categories to Spec (1) is motivated by the paper’s Table 4: Modality (96%) and Organ (83.8%) are already well-handled by the LoRA baseline, while Condition (18.5%) and Position (31.3%) are catastrophically bad. The specialized Spec pool is the scarce resource that should be directed toward the hard cases.

`__init__(self, embed_dim, hidden_dim)`

```
try:
    from sentence_transformers import SentenceTransformer
    self.encoder = SentenceTransformer(_ENCODER_MODEL)
    for p in self.encoder.parameters():
        p.requires_grad = False
except ImportError:
    self.encoder = None
```

The `try/except ImportError` is a lazy import guard: if `sentence_transformers` is not installed (e.g. during a lightweight test or A1 ablation), `self.encoder = None` and the object can still be constructed without crashing. An error is raised only when `forward()` is actually called. The `for p in self.encoder.parameters(): p.requires_grad = False` freezes the 110M encoder weights completely. They will never receive gradients. This serves two purposes: (1) keeps VRAM usage bounded, (2) provides stable, consistent embeddings throughout training — the MLP trains against a fixed feature space.

```
self.mlp = nn.Sequential(
    nn.Linear(embed_dim, hidden_dim),   # 768 → 256
    nn.ReLU(),
    nn.Dropout(0.1),
    nn.Linear(hidden_dim, 1),           # 256 → 1
    nn.Sigmoid(),                        # → (0, 1)
)
```

A two-layer MLP with 768 → 256 → 1 architecture. The capacity is intentionally small: the classification boundary between Gen and Spec questions is low-dimensional (you only need to recognize 5 question types). Dropout(0.1) provides mild regularization. The Sigmoid activation at the end is what produces `alpha ∈ (0, 1)` — it never reaches exactly 0 or 1, which is by design: you want soft blending during training, not hard switching, because hard switching makes the prompt pools fail to share gradients from cross-pool samples.

Total trainable parameters in the MLP: `768×256 + 256 + 256×1 + 1 = 197,121` — negligible compared to the ViT or LLM.

`forward(self, questions: list) → torch.Tensor`

```
device = next(self.mlp.parameters()).device

with torch.no_grad():
    embeddings = self.encoder.encode(
        questions,
        convert_to_tensor=True,
        show_progress_bar=False,
        device=str(device),
    )  # [B, 768]

alpha = self.mlp(embeddings.to(device)).squeeze(-1)   # [B]
return alpha
```

`next(self.mlp.parameters()).device` is a robust way to find the device the MLP lives on without hardcoding `"cuda"`. The sentence-transformer `encode()` method runs inside `torch.no_grad()` so no gradients are accumulated through the ~110M encoder weights. `convert_to_tensor=True` returns a PyTorch tensor rather than a numpy array. `device=str(device)` ensures the embeddings are produced on the right device directly (CUDA → CUDA) rather than CPU → CUDA transfer.

The `embeddings` tensor has shape `[B, 768]`. After `.to(device)` (ensures consistent device placement) and MLP forward, we get `[B, 1]`. `.squeeze(-1)` removes the trailing dimension to give `[B]`. Each element is a scalar in `(0, 1)` representing the routing weight for that sample.

Gradient flow: autograd traces through `self.mlp(embeddings)` because `self.mlp` has `requires_grad=True` parameters. It does NOT trace back through `self.encoder.encode()` because that call is wrapped in `torch.no_grad()`. So backpropagation updates only the 197K MLP parameters.

`category_label(category: str) → int` (static method)

```
return CATEGORY_TO_LABEL.get(category.lower(), 1)
```

A lookup with a safe default of 1 (Spec) for unknown categories. Used in two places: (1) `HierarchicalVPTModel.forward()` when computing oracle alpha for ablation A5 — maps ground-truth category strings to hard 0.0/1.0 alpha values; (2) `evaluate_router()` in `train_vpt.py` when computing router accuracy on the validation set.

3.3 `src/vpt/vpt_model.py` — `HierarchicalVPTModel`

Theoretical background. This class is an architectural wrapper that composes three separately-motivated components — a frozen ViT (adapted by VPT), a learned router, and a LoRA-adapted LLM — into one `nn.Module` that behaves like a standard HuggingFace model from the outside. The design principle is that the training loop should not need to know which component is doing what; it just calls `model(...)` and gets back `(outputs, alpha)`.

The LoRA background. LoRA (Hu et al., 2021) inserts two small matrices `A ∈ ℝ^{d×r}` and `B ∈ ℝ^{r×d}` alongside each linear layer `W ∈ ℝ^{d×d}` of the LLM. The effective weight becomes `W + (α/r) · BA`. Only A and B are trained; W stays frozen. With rank `r=16` and a 7B-parameter LLM, LoRA reduces the trainable parameter count from ~7B to ~20M — a 350× reduction — while achieving near full fine-tuning quality on task-specific data. The `lora_alpha=32` scaling factor (note: this is a different α from the routing weight — naming collision in the literature) normalizes the learning rate effectively equivalent to `lr * (lora_alpha / lora_rank) = lr * 2`, a common scaling trick.

`__init__` — full construction sequence

```
self.disable_router = disable_router
torch_dtype = torch.bfloat16 if torch.cuda.is_available() else torch.float32
self.processor = AutoProcessor.from_pretrained(model_name)
base_model = AutoModelForCausalLM.from_pretrained(
    model_name, torch_dtype=torch_dtype, device_map=device_map,
)
```

`AutoProcessor` wraps both the tokenizer (for text) and the image processor in one object. For Qwen2.5-VL it is a `Qwen2VLProcessor`; for MedGemma it is a `PaliGemmaProcessor`. The single `processor` object handles both modalities, which is why both `image` and `text` are passed to `processor(...)` simultaneously in the dataset. `torch_dtype=torch.bfloat16` loads all 7B (or 4B) parameters in 16-bit brain float, halving

VRAM compared to float32 without meaningful precision loss for inference and training (bfloat16 has the same exponent range as float32).

```
if add_lora and lora_config is not None:
    from peft import get_peft_model, LoraConfig, TaskType
    peft_cfg = LoraConfig(
        r=16, lora_alpha=32, lora_dropout=0.05,
        target_modules="all-linear",
        task_type=TaskType.CAUSAL_LM,
    )
    self.model = get_peft_model(base_model, peft_cfg)
else:
    self.model = base_model
```

`target_modules="all-linear"` is a PEFT shorthand that applies LoRA to every `nn.Linear` module in the model — including the attention Q/K/V/O projections and the feed-forward layers — in both the LLM and the visual merger. This is more aggressive than the paper's original LLaMA-Factory config (which targets only attention), but produces better results in practice. `TaskType.CAUSAL_LM` tells PEFT the model's task so it applies appropriate forward-pass modifications.

After `get_peft_model`, `self.model` is a `PeftModel` object. Calling `self.model.named_parameters()` will include both the frozen base parameters AND the trainable LoRA A/B matrices. The LoRA parameters are identifiable by the string `"lora"` appearing in their name.

```
self._freeze_visual_encoder()
vit_target = self._unwrap_peft(self.model)
self.prompt_learner = VPTPromptLearner(vit_model=vit_target, ...)
self.prompt_learner.register_hooks(vit_target)
self.router = None if disable_router else CategoryRouter()
```

Note the order: freeze happens before prompt learner is built. This matters because `_freeze_visual_encoder()` calls `requires_grad_(False)` on all visual parameters. If `VPTPromptLearner` were instantiated first and its hooks registered, the freezing would not affect the hooks (hooks don't care about `requires_grad`), but the intent is to be explicit that ViT weights are frozen. The `vit_target = _unwrap_peft(self.model)` call is explained next.

`_unwrap_peft(model) → nn.Module` (static)

```
if hasattr(model, "base_model") and hasattr(model.base_model, "model"):
    return model.base_model.model
return model
```

When PEFT wraps a model with `get_peft_model(base_model, cfg)`, the resulting `PeftModel` has this structure:

```
PeftModel
  .base_model = LoraModel
    .model    = original Qwen2_5_VLForConditionalGeneration
```

PEFT implements `__getattr__` forwarding so that `peft_model.visual` resolves correctly to `peft_model.base_model.model.visual`. However, when you call `block.register_forward_pre_hook(...)` on a block object obtained via `peft_model.visual.blocks[i]`, the hook is registered on the actual underlying block module. If you later call `peft_model2.base_model.model.visual.blocks[i]`, you'd get the same module object (Python object identity). So hook registration is unambiguous. The unwrapping is done for clarity and to ensure `_count_vit_layers` and `_get_vit_blocks` run against the concrete model class rather than the PEFT proxy.

`_freeze_visual_encoder(self)`

```
target = self._unwrap_peft(self.model)
if hasattr(target, 'visual'):            # Qwen2.5-VL
    for p in target.visual.parameters():
        p.requires_grad = False
elif hasattr(target, 'vision_tower'):    # MedGemma
    for p in target.vision_tower.parameters():
        p.requires_grad = False
```

Sets `requires_grad=False` on every ViT weight. This means: (1) no gradient is computed for ViT weights during backward pass, saving compute and memory; (2) the optimizer has no ViT parameters in any of its param groups, so AdamW momentum state for the ViT is never allocated (saves more memory). The VPT prompt tokens are NOT part of the visual encoder — they are owned by `VPTPromptLearner` which is a sibling attribute. Freezing the encoder does not affect the prompt tokens.

`_get_vit_hidden_dim(self) → int`

```
target = self._unwrap_peft(self.model)
if hasattr(target, 'visual'):
    return target.visual.config.hidden_size
return target.vision_tower.config.hidden_size
```

Reads the ViT's embedding dimension from its `config` object. For Qwen2.5-VL-7B this is 1280. For MedGemma-4B's SigLIP encoder it is 1152. This value is passed to `VPTPromptLearner` as `prompt_dim` — the width of each prompt token vector. It must match the ViT's hidden size exactly, because the prompt tokens are prepended to the hidden state tensor and must be compatible for self-attention.

`forward(self, pixel_values, input_ids, attention_mask, questions, labels, oracle_categories, image_grid_thw) → (outputs, alpha)`

```
# Step 1: compute routing weight alpha
if oracle_categories is not None:
    alpha = torch.tensor([1.0 if CategoryRouter.category_label(c)==0 else 0.0
                          for c in oracle_categories],
                         device=pixel_values.device, dtype=torch.float32)
elif self.disable_router or self.router is None:
    alpha = None
else:
    alpha = self.router(questions).to(pixel_values.device)
```

Three paths: - **Oracle (A5)**: ground-truth categories are available; alpha is hard-switched (1.0 = fully Gen, 0.0 = fully Spec). No MLP involved. This establishes an upper bound on what perfect routing could achieve. - **No router (A1)**: alpha is `None`; `_inject_tokens` will do 0.5/0.5 blending. - **Learned router (A2–A4)**: the sentence encoder + MLP produces a continuous alpha per sample. `.to(pixel_values.device)` moves from the MLP's device (same GPU, but just in case) to wherever the pixels are.

```
# Step 2: arm the hooks with alpha before the ViT runs
self.prompt_learner.set_routing_weight(alpha)
```

This must happen *before* `self.model(...)`. The hooks fire during the ViT's forward pass, which happens inside `self.model(pixel_values=..., ...)`. If `set_routing_weight` were called after, the hooks would use whatever stale alpha was left from the previous call.

```
# Step 3: VLM forward (ViT + visual merger + LLM, in sequence internally)
forward_kwargs = dict(pixel_values=pixel_values, input_ids=input_ids,
                      attention_mask=attention_mask, labels=labels)
if image_grid_thw is not None:
```

```
    forward_kwargs["image_grid_thw"] = image_grid_thw

return self.model(**forward_kwargs), alpha
```

`image_grid_thw` is a Qwen2.5-VL-specific tensor encoding the grid dimensions `(temporal, height, width)` of the image's patch layout. It is needed by the model's 3D rotary position embedding (RoPE-3D). MedGemma does not use it. By only including it when present, the same `forward()` signature works for both models.

The return is a tuple `(CausalLMOutputWithPast, alpha_tensor)`. `outputs.loss` is the language-model cross-entropy on the masked labels. `alpha` is the routing weight — returned so the training loop can use it for the router loss terms.

`trainable_parameters(self) → list`

```
params = (
    list(self.prompt_learner.gen_prompts.parameters()) +
    list(self.prompt_learner.spec_prompts.parameters()) +
    list(self.router.mlp.parameters()) +
    [p for p in self.model.parameters() if p.requires_grad]
)
```

An explicit enumeration of all parameters that should receive gradient updates. In practice, the training loop does not call this method — it builds separate param groups directly. But this method exists for verification: you can call it and compare to the optimizer's param groups to confirm nothing was missed or double-counted.

The filter `[p for p in self.model.parameters() if p.requires_grad]` picks up the LoRA A/B matrices while excluding the frozen base weights (including the frozen ViT). This is because `self.model.parameters()` returns everything inside the PEFT-wrapped model, but only the LoRA parameters have `requires_grad=True`.

`print_trainable_summary(self)`

```
total    = sum(p.numel() for p in self.parameters())
trainable= sum(p.numel() for p in self.parameters() if p.requires_grad)
print(f"Total parameters:     {total:,}")
print(f"Trainable parameters: {trainable:,} ({100*trainable/total:.2f}%)")
```

Printed once at the start of training. For A4 on Qwen2.5-VL-7B, expect approximately: - Total: ~7.6B (7B LLM + 600M ViT + small VPT/router) - Trainable: ~22M LoRA + ~1.5M VPT prompts + ~0.2M router MLP ≈ 23.7M (~0.31%)

This is the key efficiency claim: adapting less than 0.5% of parameters to shift Condition/Position performance by double-digit percentages.

3.4 `train_vpt.py` — VPT trainer (detailed)

`BanglaMedVQADataset.__init__`

```
with open(json_path, encoding="utf-8") as f:
    data = json.load(f)
if max_samples is not None:
    data = data[:max_samples]
self.data = data

df = pd.read_csv(csv_path)
self.cat_lookup = dict(zip(df["question_bn"].astype(str), df["category"].astype(s
        tr)))
```

The JSON is a ShareGPT-format list: each element is `{"conversations": [...], "images": [...]}`. The CSV carries metadata including the `category` column (`modality`, `organ`, `abnormality`, `condition`, `position`). These are joined on `question_bn` to build `cat_lookup`, a dict mapping Bangla question text to category string. This is the category used for router supervision loss and for router accuracy evaluation. The `max_samples` parameter enables the `--debug` mode to run on just 10 examples.

`BanglaMedVQADataset.__getitem__` — label masking logic

This is the most nuanced part of the dataset class. Understanding it requires understanding how causal language model training works.

Background on causal LM labels. A causal LM outputs, for each position `t`, a probability distribution over the vocabulary for the next token. The loss for one sample is the average negative log-probability of the correct next token at each position. For a sequence `[t1, t2, t3, t4, t5]`, the loss is:

```
L = -log P(t2|t1) - log P(t3|t1,t2) - log P(t4|t1,t2,t3) - log P(t5|t1,t2,t3,t4)
```

In practice, HuggingFace implements this by asking you to pass a `labels` tensor of the same length as `input_ids`, where each position in `labels` is the *target* for the prediction at that position. The special value `-100` means "ignore this position in the loss." So to train the model to produce only the answer (not to reproduce the question prefix), you set all label positions corresponding to the question/system-prompt to `-100`.

```
prefix_msgs = [{"role": "user", "content": [
    {"type": "image"}, {"type": "text", "text": question},
]}]
prefix_text = self.processor.apply_chat_template(prefix_msgs,
        add_generation_prompt=True)
full_text   = prefix_text + answer + self.processor.tokenizer.eos_token
```

`apply_chat_template` converts the message list into the model's native chat format. With `add_generation_prompt=True`, it appends the assistant's turn prefix (e.g. `<|im_start|>assistant\n` for Qwen) so the sequence ends exactly where the answer should begin. `full_text` is then `[system + user_turn + assistant_prefix + answer + EOS]`.

```
full_inputs = self.processor(
    text=[full_text], images=[image],
    return_tensors="pt", padding="max_length", truncation=True,
    max_length=self.max_length, add_special_tokens=False,
)
```

A single processor call handles both image and text. The processor: (1) encodes the image into pixel patches and produces `pixel_values`; (2) tokenizes `full_text` into `input_ids`; (3) for Qwen2.5-VL, also produces `image_grid_thw`. `padding="max_length"` pads to exactly 512 tokens, ensuring all samples in a batch have the same length and can be stacked into a rectangular tensor without a custom collator for the tensor parts. `add_special_tokens=False` avoids the chat-template tokens being added *again* by the tokenizer (the template already includes them via `apply_chat_template`).

```
prefix_text_ids = self.processor.tokenizer(
    prefix_text, return_tensors="pt",
    add_special_tokens=False, truncation=True, max_length=self.max_length,
).input_ids[0]
prefix_len = prefix_text_ids.shape[0]
```

To find the boundary between question and answer, tokenize just the prefix text (without the image, which adds special placeholder tokens that shift the count). The tokenizer is deterministic — the same string always tokenizes to the same tokens — so `prefix_len` is the number of text tokens in the prefix. Everything from position `prefix_len` onward in `input_ids` corresponds to the answer text.

```
labels = input_ids.clone()
labels[:prefix_len] = -100
labels[attention_mask == 0] = -100
```

Clone `input_ids` into `labels`, then mask out: (1) the question prefix positions (`[:prefix_len]`), and (2) all padding positions (`attention_mask == 0`). Result: only the

answer token positions retain their actual token IDs; everywhere else is -100. The LM loss will sum only over those retained positions, computing how well the model predicts the answer tokens given the full prefix context.

`collate_fn(batch: list) → dict`

```
for k in keys:
    if isinstance(batch[0][k], torch.Tensor):
        result[k] = torch.stack([b[k] for b in batch])
    else:
        result[k] = [b[k] for b in batch]
```

PyTorch's default collator cannot handle `str` values (the question text and category string) in a batch. This custom function stacks tensors into batches normally and passes string values through as Python lists. Those lists are what `CategoryRouter.forward(questions: list)` expects.

`compute_loss(lm_loss, router_alpha, category_labels, lambda_router, mu_cat) → Tensor`

This function implements the three-term loss that simultaneously teaches the LM to produce correct Bangla answers AND teaches the router to correctly classify questions.

Term 1 — Language Model loss:

```
L_lm = outputs.loss
```

This is the standard causal LM cross-entropy computed by HuggingFace internally. It measures how well the model predicts the correct answer tokens given the image and question prefix.

Term 2 — Router entropy regularisation:

```
probs    = torch.stack([router_alpha, 1.0 - router_alpha], dim=-1).clamp(1e-8, 1.
         0)
L_entropy = -(probs * probs.log()).sum(dim=-1).mean()
L_router  = -lambda_router * L_entropy
```

`router_alpha` is a `[B]` tensor. Stacking with `1 - router_alpha` gives a `[B, 2]` probability distribution. The binary entropy is `H[p] = -p·log(p) - (1-p)·log(1-p)`. Averaging over the batch gives a scalar entropy. The loss adds `-lambda_router * H` (negative entropy, because we maximise entropy to get a negative loss term). Why do we want to *maximise* entropy? Because if the router always outputs alpha ≈ 0.0 or ≈ 1.0 for

everything, one pool never gets gradient signal and its parameters stay near their initialization. The entropy regulariser discourages the router from collapsing to a degenerate solution where it always picks the same pool.

Term 3 — Supervised category classification:

```
cat_targets = torch.tensor([CATEGORY_TO_LABEL.get(c.lower(), 1) for c in
        category_labels],
                                dtype=router_alpha.dtype, device=router_alpha.device)
L_cat = F.binary_cross_entropy(router_alpha, cat_targets)
```

`cat_targets` is a `[B]` tensor of 0s and 1s. `binary_cross_entropy(alpha, target)` is `-target·log(alpha) - (1-target)·log(1-alpha)` — the standard binary classification loss. This is the primary supervision signal for the router: it pushes `alpha` toward 1.0 for Gen questions (modality/organ) and toward 0.0 for Spec questions (abnormality/condition/position). Without this term, the router would only be guided by the downstream LM loss (which is noisy and indirect) and the entropy regulariser (which only prevents collapse, not correct classification).

Combined:

```
L_total = L_lm + (-0.01 * H[alpha]) + 0.1 * BCE(alpha, target)
```

The weights `lambda_router=0.01` and `mu_cat=0.1` ensure the LM loss (which is typically in the range 1–4 nats) dominates. The classification loss is scaled down because a perfectly trained router is not necessary for good VQA performance — the routing is a means to an end, not the end itself.

`evaluate_router(model, loader, device) → float`

```
alpha = model.router(questions).to(device)
preds = (alpha < 0.5).long()   # alpha >= 0.5 → predict 0 (Gen); < 0.5 → predict
        1 (Spec)
targets = torch.tensor([CATEGORY_TO_LABEL.get(c.lower(), 1) for c
        in categories], device=device)
correct += (preds == targets).sum().item()
```

Threshold alpha at 0.5: samples where the router outputs alpha ≥ 0.5 are predicted as Gen (label 0); alpha < 0.5 are predicted as Spec (label 1). Compare to ground-truth binary labels. Returns a float in `[0, 1]` representing classification accuracy. A random router would score ~0.6 (because the BanglaMedVQA dataset has ~60% Spec questions). Meaningful is ≥ 0.8. The `@torch.no_grad()` decorator disables gradient tracking for the entire function — no gradients are needed for evaluation, and excluding them saves memory.

`train(args)` — the training loop

Model and data setup:

```
lora_cfg = {"lora_rank": 16, "lora_alpha": 32, "lora_dropout": 0.05}
device_map = "auto" if device != "cpu" else {"": "cpu"}
model = HierarchicalVPTModel(...)
model.print_trainable_summary()
if args.gradient_checkpointing:
    model.model.gradient_checkpointing_enable()
```

`gradient_checkpointing_enable()` is a memory/compute trade-off: during the forward pass, intermediate activations are not stored (to save VRAM). During backward, they are recomputed on-the-fly. On an A100 80 GB, this typically allows a 40–50% larger batch or model. The hooks still fire correctly during the recomputed forward because `_current_alpha` is set per training step before any model forward and remains constant until the next step.

Optimiser with separate learning rates:

```
vpt_params    = list(gen_prompts.parameters()) + list(spec_prompts.parameters())
router_params = list(model.router.mlp.parameters()) if model.router else []
lora_params   = [p for n, p in model.model.named_parameters() if "lora" in
        n.lower() and p.requires_grad]

param_groups = [{"params": vpt_params, "lr": 2e-4}]
if router_params: param_groups.append({"params": router_params, "lr": 1e-3})
if lora_params:   param_groups.append({"params": lora_params,   "lr": 1e-4})
optimizer = torch.optim.AdamW(param_groups, weight_decay=0.01)
```

Separate learning rates are important because the three components have very different learning dynamics: - **VPT prompts** start from random small values; they need `2e-4` to learn meaningful patterns in the first epoch. - **Router MLP** is a simple binary classifier; `1e-3` gets it converging fast. - **LoRA adapters** are modifying a pre-trained LLM; `1e-4` prevents them from diverging and forgetting useful language priors.

All three groups share `weight_decay=0.01` (L2 regularisation on weights, not applied to biases by AdamW's default). Empty groups are silently dropped (a guard is added before appending to `param_groups`).

```
total_steps  = len(train_loader) * args.num_epochs
warmup_steps = max(1, int(total_steps * 0.05))
scheduler = get_cosine_schedule_with_warmup(optimizer, warmup_steps, total_steps)
```

Cosine annealing with 5% linear warmup. Warmup prevents large parameter updates early in training when the loss landscape is unfamiliar. After warmup, cosine decay gradually reduces the LR to near zero by the final step.

Two-phase training:

```
if epoch == 0:
    for p in lora_params: p.requires_grad_(False)  # Phase 1: freeze LoRA
elif epoch == 1:
    for p in lora_params: p.requires_grad_(True)   # Phase 2: unfreeze
```

Phase 1 (epoch 1) trains only the VPT prompt tokens and router MLP while LoRA stays frozen. The rationale: if LoRA is active from the start, it and the VPT tokens are competing to fit the training data and neither may find a good solution. By letting VPT establish a useful visual representation first, Phase 2 starts with a better visual prior for LoRA to build on.

Inner loop:

```
outputs, router_alpha = model(pixel_values, input_ids, attention_mask, questions,
                              labels=labels, oracle_categories=oracle_cats, ...)

if router_alpha is None or args.use_oracle_router:
    loss = outputs.loss
else:
    loss = compute_loss(outputs.loss, router_alpha, categories, ...)

optimizer.zero_grad()
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
optimizer.step()
scheduler.step()
```

`clip_grad_norm_(model.parameters(), 1.0)` clips the global gradient norm to 1.0 before the parameter update. This prevents any single batch (e.g. a very unusual image) from causing a massive parameter update that wrecks the model. With bfloat16 and hooks, gradient norms can occasionally spike without clipping.

Checkpoint saving:

```
ckpt_payload = {
    "gen_prompts":    model.prompt_learner.gen_prompts.detach().cpu(),
    "spec_prompts":   model.prompt_learner.spec_prompts.detach().cpu(),
    "num_gen_tokens": ..., "num_spec_tokens": ..., "epoch": ..., "args": ...,
}
if model.router is not None:
    ckpt_payload["router_mlp"] = model.router.mlp.state_dict()
torch.save(ckpt_payload, os.path.join(ckpt_dir, "vpt_router.pt"))
if args.add_lora:
    model.model.save_pretrained(ckpt_dir)  # saves adapter_config.json + adapter
        weights
```

`.detach().cpu()` on the prompt tensors: moves to CPU before saving to avoid re-loading the checkpoint requiring a GPU. The LoRA adapter is saved with PEFT's

`save_pretrained()` which writes `adapter_config.json` (JSON metadata about LoRA rank/alpha/target modules) and `adapter_model.safetensors` (the A/B weight matrices). Both files live in the same `checkpoint-epochN/` directory alongside `vpt_router.pt`.

`_load_yaml_config(path: str) → dict`

```
cfg = yaml.safe_load(f) or {}
if not isinstance(cfg, dict): raise ValueError(...)
return cfg
```

`yaml.safe_load` parses the YAML and returns a Python dict. `or {}` handles empty YAML files gracefully. The `isinstance` check guards against YAML that evaluates to a scalar or list (malformed configs).

YAML merge logic in `main()`

```
import sys
yaml_cfg = _load_yaml_config(args.config)
cli_provided = set()
for tok in sys.argv[1:]:
    if tok.startswith("--"):
        cli_provided.add(tok.lstrip("-").replace("no-", "").split("=")[0])
for k, v in yaml_cfg.items():
    if k not in cli_provided and hasattr(args, k):
        setattr(args, k, v)
```

`sys.argv[1:]` is the raw command-line token list. We identify which flags were explicitly provided on the CLI by looking for `--`-prefixed tokens, stripping prefix and `no-` (for BooleanOptionalAction flags like `--no-add_lora`). Any YAML key that is NOT in `cli_provided` (i.e., the user didn't explicitly pass that flag) and that exists as an attribute in `args` (i.e., is a recognized argument) gets overwritten with the YAML value. CLI always wins.

3.5 `infer_vpt.py` — VPT inference (detailed)

`load_vpt_checkpoint(checkpoint_dir, model_name, add_lora, disable_router, device)`

```
payload = torch.load(ckpt_file, map_location="cpu")
num_gen_tokens  = int(payload.get("num_gen_tokens",  payload["gen_prompts"].shape
        [1]))
num_spec_tokens = int(payload.get("num_spec_tokens", payload["spec_prompts"].sha
        pe[1]))
```

`map_location="cpu"` loads the checkpoint onto CPU first regardless of where training was run. This avoids having two GPU copies of the prompt tensors simultaneously. The fallback `payload["gen_prompts"].shape[1]` infers token count from the tensor shape for older checkpoints that didn't save the count explicitly.

```
model = HierarchicalVPTModel(model_name=..., lora_config=lora_cfg if add_lora el
        se None, ...)
with torch.no_grad():
    model.prompt_learner.gen_prompts.copy_(payload["gen_prompts"].to(...dtype...)
        )
    model.prompt_learner.spec_prompts.copy_(payload["spec_prompts"].to(...dtype..
        .))
if model.router is not None and "router_mlp" in payload:
    model.router.mlp.load_state_dict(payload["router_mlp"])
if add_lora:
    model.model.load_adapter(checkpoint_dir, adapter_name="default")
```

`copy_()` is an in-place tensor copy that respects the existing dtype of the parameter (bfloat16 on GPU). `load_state_dict` for the router MLP loads the 197K parameters from the checkpoint. `load_adapter(checkpoint_dir)` is PEFT's method that reads `adapter_config.json` and `adapter_model.safetensors` from the directory and merges them into the already-instantiated PEFT model.

`run_inference(model, test_data, cat_lookup, ...)` — the generation loop

```
model.prompt_learner.set_routing_weight(alpha)
output_ids = model.model.generate(**inputs, max_new_tokens=128, do_sample=False,
        ...)
pred = model.processor.decode(output_ids[0][inputs["input_ids"].shape[1]:],
                              skip_special_tokens=True).strip()
```

`set_routing_weight(alpha)` must be called before `model.model.generate()` for the same reason as during training: the hooks fire inside the ViT pass that happens at the start of every `generate()` step. `generate()` runs autoregressively: it calls the model's forward

pass repeatedly, each time appending the top-1 predicted token to the sequence. The ViT runs only once (on the first call); subsequent calls skip it.

`output_ids[0][inputs["input_ids"].shape[1]:]` slices off the input prefix from the generated token sequence, keeping only the newly generated tokens. `skip_special_tokens=True` removes EOS/BOS/pad tokens from the decoded string.

The `alpha_val` column in the output CSV records the router's routing decision per sample — useful for analysis (e.g. checking what the router assigns to ambiguous questions) and for correlation with per-sample performance.

3.6 `evaluate.py` — per-category metrics (detailed)

`compute_accuracy(df) → dict`

```
df["match"] = (df["predicted_answer_bn"].str.strip().str.lower() ==
               df["llm_answer_bn"].str.strip().str.lower())
```

Exact string match after lowercasing and stripping whitespace. This is deliberately strict: "হ্যাঁ" and "হ্যাঁ" (trailing space) do not match. Exact match is appropriate for short, closed-form VQA answers ("Yes", "No", "Pneumonia") but will undercount partial credit for longer answers. That is why BERTScore and LAVE are added alongside it.

`compute_bertscore(df) → dict`

```
_, _, F1 = bert_score_fn(preds, refs, lang="bn", verbose=False)
```

BERTScore (Zhang et al., 2020) computes token-level cosine similarity between contextualized embeddings of prediction and reference, then takes precision, recall, and F1 over the greedy token matching. Specifying `lang="bn"` selects `xlm-roberta-large` as the scoring model — a 560M-parameter model trained on 100 languages including Bangla. This metric captures semantic similarity that exact match misses (e.g. a different valid phrasing of the same diagnosis).

`compute_lave(df, openai_client) → dict`

```
prompt = (
    "You are a strict evaluator for a medical VQA task in Bangla.\n"
    "For each pair below, output ONLY a JSON list of floats in [0,1].\n"
    "Score 1.0 if semantically equivalent, 0.5 if partially correct, 0.0 if
        wrong.\n"
```

```
    ...
)
response = openai_client.chat.completions.create(model="gpt-4.1-mini", ...)
batch_scores = json.loads(response.choices[0].message.content.strip())
```

LAVE (LLM-Assisted VQA Evaluation, Mañas et al. 2024) uses a strong LLM as a judge. Pairs are batched in groups of 20 per API call to minimize latency and cost. GPT-4.1-mini is instructed to output a raw JSON array — no markdown, no explanation — because `json.loads` would fail on anything else. The `try/except` catches both API errors (rate limits, timeouts) and JSON parse failures (model produces unexpected output), defaulting the batch to `0.0` in those cases.

3.7 `train_model.py` — LoRA baseline (B0, detailed)

The file has three logical parts:

`MedVQADataset`: reads a CSV and filters rows with missing image paths, questions, or answers. The `get_dataset_paths()` method constructs conventional paths from a dataset name + split, e.g. `data/chest_x-ray/train/chest_x-ray.csv`.

`MedVQATrainer`: wraps a YAML config file (the LLaMA-Factory format, distinct from the VPT YAML format) and decides from config keys whether to run in `train` or `test` mode. `train()` delegates to `llamafactory-cli train <config>`. `test()` delegates to `run_real_inference()`.

`run_real_inference()`: this is where B0 predictions are generated. Key implementation detail:

```
cat_lookup = dict(zip(df_csv["question_bn"].astype(str), df_csv["category"].asty
        pe(str)))
...
predictions_data.append({
    ...
    "category": cat_lookup.get(question, ""),
    ...
})
```

This was the fix applied during the reviewer pass. Previously this field was `''`, making the output CSV unusable by `evaluate.py`. Now it is populated from the original CSV, enabling per-category evaluation of the B0 baseline — which is necessary to confirm that A4 actually improves Condition and Position without degrading Modality and Organ.

---

4. Data flow — detailed, with tensor shapes

```
────────────────────────────────────────────────────────────────────────────────
TRAINING STEP (one batch, B = batch_size = 4)
────────────────────────────────────────────────────────────────────────────────

 CSV + JSON files
     │
     ▼  BanglaMedVQADataset.__getitem__
 per sample:
   image          PIL RGB
   full_text      "...prefix...answer<EOS>"  [string]
   question       Bangla question             [string]
   category       "condition" / "organ" / …  [string]
     │
     ▼  processor(text=full_text, images=image)
   pixel_values     [C, H, W] or [1, P, D] depending on model
   input_ids        [512]  (padded)
   attention_mask   [512]
   image_grid_thw   [3] (Qwen2.5-VL only)
   labels           [512]  (-100 everywhere except answer positions)
     │
     ▼  collate_fn stacks B samples
   pixel_values     [B, C, H, W]
   input_ids        [B, 512]
   attention_mask   [B, 512]
   labels           [B, 512]
   questions        list[str, B]
   categories       list[str, B]
     │
     ▼  HierarchicalVPTModel.forward
     │
     ├── CategoryRouter.forward(questions)
     │     encoder.encode()  →  [B, 768]  (frozen, no grad)
     │     mlp(embeddings)   →  [B]       alpha in (0,1), float32, has grad
     │     set_routing_weight(alpha) stores [B, 1, 1] on prompt_learner
     │
     └── self.model(pixel_values, input_ids, attention_mask, labels, …)
           │
           ▼  ViT forward (Qwen2.5-VL: 32 blocks)
           │
           Block 0 pre-hook (_inject_tokens, layer_idx=0):
             hidden_in:  [B, 1+P, 1280]       (1 CLS + P patches, D=1280)
             blended:    [B, 24, 1280]         (N=max(16,24) prompts, bfloat16)
             hidden_out: [B, 25+P, 1280]       (N prompts prepended)
             return (hidden_out, ...)
           │
           Block 0 self-attention processes [B, 25+P, 1280]
           │
           Block 1 pre-hook (_inject_tokens, layer_idx=1):
             hidden_in:  [B, 25+P, 1280]       (from block 0 output)
             strip:      hidden_in[:, 24:, :]  → [B, 1+P, 1280]
             new blend:  [B, 24, 1280]         (layer 1 prompt params)
             hidden_out: [B, 25+P, 1280]
```

```
            |
            ... (blocks 2–30 follow the same strip + inject pattern)
            |
            Block 31 pre-hook:   strip + inject as above
            Block 31 self-attn:  [B, 25+P, 1280]
            Block 31 post-hook (_strip_tokens):
              output[:, 24:, :]  →  [B, 1+P, 1280]   (prompts removed)
            |
            ▼  visual merger (e.g. MLP projection)
            [B, 1+P, 1280] → [B, 1+P, 3584]  (Qwen2.5-VL LLM width)
            |
            ▼  LLM (32 transformer blocks, LoRA active)
            [B, 512 + 1+P, 3584]  (visual tokens prepended to text tokens)
            |
            ▼  LM head → logits → cross-entropy on labels
            outputs.loss   scalar (float32, has grad through LoRA)
      |
      ▼  compute_loss(outputs.loss, alpha, categories)
        L_lm      = outputs.loss
        L_router  = -0.01 · H[alpha]   (entropy reg)
        L_cat     = 0.1 · BCE(alpha, target)
        loss      = L_lm + L_router + L_cat   (scalar, float32)
      |
      ▼  loss.backward()
        Gradients flow into:
          - gen_prompts / spec_prompts  (via the hook tensors)
          - router.mlp weights          (via alpha → L_router + L_cat)
          - LoRA A/B matrices           (via outputs.loss, Phase 2 only)
      |
      ▼  optimizer.step() → parameters updated
      ▼  scheduler.step() → LR decayed
──────────────────────────────────────────────────────────────────────────────
INFERENCE STEP (infer_vpt.py, one sample at a time)
──────────────────────────────────────────────────────────────────────────────


 Load checkpoint: gen_prompts, spec_prompts, router_mlp, LoRA adapter
      |
      ▼  per test sample
    image, question, ground_truth = from JSON
    category = from CSV lookup
      |
      ▼  compute alpha (oracle or router or None)
    prompt_learner.set_routing_weight(alpha)
      |
      ▼  model.model.generate(pixel_values, input_ids, max_new_tokens=128)
            ViT runs once with VPT hooks → visual features
            LLM generates token by token (128 max)
      |
      ▼  decode output_ids, strip prefix
    predicted_answer = str
      |
      ▼  write CSV row: image_id, category, llm_answer_bn, predicted_answer_bn,
alpha
──────────────────────────────────────────────────────────────────────────────
EVALUATION (evaluate.py)
──────────────────────────────────────────────────────────────────────────────
```

```
predictions.csv  →  compute_accuracy()   →  exact match per category
                 →  compute_bertscore()  →  F1 via xlm-roberta per category
                 →  compute_lave()       →  GPT-4.1-mini judge per category
                 →  JSON results file + terminal table
```

5. Hyperparameters — with rationale

Parameter	Value	Why this value
`num_gen_tokens`	16 per layer	Enough capacity for broad image understanding without over-parameterizing the Gen pool, which handles two already-strong categories (Modality 96%, Organ 83.8%).
`num_spec_tokens`	24 per layer	1.5× Gen because Spec covers three harder categories. More tokens = more capacity to encode region-specific visual cues. The 16 vs. 24 asymmetry is the “hierarchical” part.
`num_prompt_tokens`	24 (= max)	Derived constant. Kept fixed across layers to make the strip-and-reinject sequence-length bookkeeping exact.
`lora_rank`	16	Matches the paper’s baseline. Rank 16 gives ~20M adapter parameters for a 7B LLM — well in the “diminishing returns” regime for this dataset size.
`lora_alpha`	32	`alpha/rank = 2` means the LoRA contribution is scaled by 2 at initialization. Equivalent to multiplying the LoRA learning rate by 2 vs. rank=16/alpha=16. Standard practice.
`lora_dropout`	0.05	Light regularization on the LoRA adapters. Slightly higher than the paper (0.0) because VPT introduces additional trainable parameters that could overfit jointly.
`lr_vpt`	2e-4	Prompt tokens start from random noise; they need an aggressive learning rate in Phase 1 to establish meaningful

Parameter	Value	Why this value
		representations quickly. Halved to 1e-4 implicitly in Phase 2 due to cosine decay.
`lr_router`	1e-3	The router MLP (197K params) is a simple classifier; it needs a higher LR than the prompts to converge within the first epoch.
`lr_lora`	1e-4	Standard LLM fine-tuning learning rate. Low enough to preserve pre-trained language knowledge while fitting the Bangla medical domain.
`lambda_router`	0.01	Entropy regularisation is a nudge, not a primary objective. Scaling by 0.01 keeps its contribution to the total loss below 5% even at maximum entropy.
`mu_cat`	0.1	Category BCE supervision is ~10× the entropy term. It's still secondary to `L_lm` (which is typically 1–4 nats, so `0.1 × BCE ≈ 0.05–0.1` nats).
`batch_size`	4	Limited by A100 80 GB VRAM when running A4 (7B model + LoRA + VPT + optimizer states). Gradient accumulation could increase effective batch size but is not yet implemented.
`num_epochs`	3	1 Phase 1 + 2 Phase 2. BanglaMedVQA has ~4,900 training samples; 3 epochs gives ~14,700 optimization steps at batch size 4. More epochs risk overfitting on this small dataset.
`max_length`	512	Balances the typical token count of a Bangla VQA sample (question ~30 tokens + visual tokens + answer ~10 tokens ≪ 512) against the cost of padding. Truncation at 512 should be rare.
`warmup_ratio`	0.05	5% of total steps (≈ 735 steps) for linear LR warmup. Prevents instability from large gradients when the prompts and router are uninitialized.
Sentence encoder		

Parameter	Value	Why this value
	`paraphrase-multilingual-mpnet-base-v2`	768-dimensional embeddings, trained on parallel multilingual data. Handles Bangla text without requiring a separate Bangla tokenizer or model.

---

6. Ablations — scientific design

The five ablations (A1–A5) form a clean stepwise decomposition where each step adds exactly one component. This is the standard ablation study design: you cannot claim a component contributes if you only test the full system versus a baseline. Removing components one at a time isolates each contribution.

B0 — LoRA-only baseline (the paper we reproduce)

What it is. Fine-tune Qwen2.5-VL-7B with LoRA on BanglaMedVQA using LLaMA Factory. The ViT is fully frozen. No VPT, no router.

What it tests. Whether the original paper's reported numbers are reproducible. You must verify B0 before running any A ablation, because the comparison claims ("A4 improves Condition by +20 points over B0") are only meaningful if B0 is correctly reproduced.

Expected numbers (from paper Table 4, Qwen2.5-VL-7B, Bangla): Modality 96.0% · Organ 83.8% · Abnormality 75.0% · Condition 18.5% · Position 31.3% · Overall 49.3%.

Run: `python train_model.py --config configs/qwen2.5-vl-7b.yaml`

A1 — Shared VPT, no router (`disable_router=True, add_lora=False`)

What it adds over B0. A single shared pool (both Gen and Spec tokens equally blended, alpha=0.5) injected into the ViT. No router. No LoRA.

What it tests. Whether VPT tokens alone — any VPT tokens, regardless of their specificity — provide any benefit over a frozen ViT. If A1 > B0 on Condition/Position but A1 < B0 on Modality/Organ, we know VPT is directionally useful but category-agnostic prompts steal capacity from the already-good categories.

Implementation detail. Both `num_gen_tokens` and `num_spec_tokens` are set to 20 (equal) in A1, making the two pools identical in capacity. `disable_router: true` in the YAML causes `CategoryRouter` to not be instantiated. `alpha=None` in the hook triggers

`0.5 * g + 0.5 * s` — equivalent to a single pool of 20 tokens (since g and s are padded to 20 and added with equal weight).

No LoRA because we want to see VPT's effect on the ViT independently. If we include LoRA, improvements could come from either component.

Run: `python train_vpt.py --config configs/vpt/A1_shared_no_router.yaml --gradient_checkpointing`

A2 — Shared VPT + router (`add_lora=False`)

What it adds over A1. The learned router (MLP on sentence embeddings) now outputs a per-sample alpha that blends the two 20-token pools differently for different questions. But because both pools have the same size (20 = 20), the "Gen" and "Spec" labels are just names — the model must *discover* through training that directing one pool toward Gen questions and the other toward Spec questions is useful.

What it tests. Whether routing — even with pools of equal capacity — helps. A2 > A1 would mean the router itself, not the pool asymmetry, provides benefit. A2 ≈ A1 would mean routing without an asymmetric pool design doesn't matter.

Run: `python train_vpt.py --config configs/vpt/A2_shared_with_router.yaml --gradient_checkpointing`

A3 — Split pools + router, no LoRA (core VPT contribution)

What it adds over A2. The pool sizes are now asymmetric: Gen = 16 tokens, Spec = 24 tokens. This is the "hierarchical" design: the Spec pool has 50% more capacity, encoding the bet that Condition/Position grounding requires more visual prompt complexity than Modality/Organ.

What it tests. Whether the pool asymmetry matters beyond the routing itself. A3 > A2 would mean the specific 16/24 split is beneficial, confirming the hierarchical hypothesis. A3 ≈ A2 would suggest that the routing (not the capacity asymmetry) is the dominant factor.

Still no LoRA to keep the visual adaptation effect isolated.

Run: `python train_vpt.py --config configs/vpt/A3_split_with_router.yaml --gradient_checkpointing`

A4 — Full proposed method: split pools + router + LoRA

What it adds over A3. Re-enables LoRA on the LLM with the same configuration as B0. Now both the visual encoder (via VPT) and the language model (via LoRA) are adapted jointly, in the two-phase training schedule.

What it tests. Whether VPT and LoRA are complementary. The hypothesis is yes: VPT fixes the visual grounding failure that LoRA alone cannot address, while LoRA maintains the language quality that VPT prompts alone cannot fix (since VPT prompts influence only the ViT, not the LLM).

Expected result. The headline claim: A4 improves Condition from 18.5% to ~38% and Position from 31.3% to ~50%, while maintaining or improving Modality and Organ.

Run: `python train_vpt.py --config configs/vpt/A4_split_router_lora.yaml --gradient_checkpointing`

A5 — Oracle router upper bound (`use_oracle_router=True`)

What it adds over A4. Replaces the learned MLP router with oracle routing: at both training and inference time, the ground-truth category label is used to compute a hard alpha (1.0 for Gen, 0.0 for Spec). The router MLP is never trained or consulted.

What it tests. The performance ceiling assuming a perfect router. If A5 >> A4, the learned router has significant room for improvement and is a bottleneck. If A5 ≈ A4, the learned router is near-optimal and the remaining gap to A5 is irreducible (due to prompt capacity or LM limitations).

Why this matters for the paper. A5 gives you the sentence for the analysis section: "Our learned router (A4) achieves X% of the oracle upper bound (A5), suggesting the routing is near-optimal and further gains require more expressive prompt pools."

Run:

```
python train_vpt.py --config configs/vpt/A5_oracle_router.yaml --add_lora --use_oracle_router --gradient_checkpointing
```

Reading the ablation table for a paper

The expected argument structure in the paper is:

1. B0 → A1: *"VPT tokens alone, without routing, yield a moderate improvement in Condition/Position (+X pt) but slightly hurt Modality/Organ (–Y pt), confirming that category-agnostic prompts share capacity suboptimally."*

2. A1 → A2: *“Adding the router while keeping equal-sized pools improves Condition/Position by +Z pt, showing that routing guidance — even without asymmetric capacity — is beneficial.”*
3. A2 → A3: *“Introducing pool asymmetry (16 Gen, 24 Spec) yields an additional +W pt, validating the hierarchical capacity allocation hypothesis.”*
4. A3 → A4: *“Joint LoRA + VPT (full system) adds +V pt by leveraging both visual and language adaptation synergistically.”*
5. A4 vs A5: *“The learned router achieves R% of the oracle upper bound, indicating the routing quality is not a bottleneck for further gains.”*

---

7. Reviewer pass — issues found and fixes applied (2026-04-27)

#	Issue	File / location
1	**Critical training bug.** `BanglaMedVQADataset` tokenised the answer separately and passed it as `labels`. The HF causal-LM loss expects `labels` aligned positionally with `input_ids`; this fed unrelated token IDs into the loss, so the LM was being optimised against effectively random targets.	`train_vpt.py::BanglaMedVQADataset.__getitem__`
2	**Critical dtype bug.** `alpha` came from the float32 router MLP; `g` / `s` were created in `hidden.dtype` (bfloat16). The blend `alpha * g + (1-alpha) * s` would either error or silently upcast.	`src/vpt/prompt_learner.py::_inject_tokens`
3	**Missing inference path.** No script existed to take a VPT checkpoint and produce the predictions CSV that `evaluate.py` requires.	new file `infer_vpt.py`

#	Issue	File / location
4	**A1 had no actual `disable_router` switch.** A1 yaml claimed “alpha=None → 0.5 blend”, but `vpt_model.forward` always called the router.	`vpt_model.py`, `train_vpt.py`, `configs/vpt/A1_*.yaml`
5	**YAML configs were inert.** `train_vpt.py` only used CLI args; the per-ablation YAML files in `configs/vpt/` were documentation.	`train_vpt.py::main`
6	**PEFT-wrapped attribute access fragility.** `self.model.visual` happened to work via PEFT’s `__getattr__` forward, but registering hooks on the wrapped vs. underlying module is brittle.	`vpt_model.py::_unwrap_peft`, `_freeze_visual_encoder`, `_get_vit_hidden_dim`, ctor
7	**Checkpoint did not record token counts.** `infer_vpt.py` needs `num_gen_tokens` and `num_spec_tokens` to rebuild the learner.	`train_vpt.py` checkpoint save
8	**Validation router-accuracy code crashed when router was `None`.**	`train_vpt.py::train`

#	Issue	File / location
9	**Stale issue from baseline.** `train_model.py::run_real_inference()` previously wrote an empty `category` column; `evaluate.py` requires it for per-category breakdown.	`train_model.py::run_real_inference`

Items deliberately **not** changed: - The two-phase schedule's hard-coded epoch boundary (Phase 1 = epoch 1) — matches the protocol; could be parameterised later if needed. - `padding="max_length"` to 512 — wastes some compute but avoids a custom collator and dynamic padding edge cases on the multimodal processor. - Hard-coded LoRA hyperparameters in `train_vpt.py` — match paper baseline; `lora_config` is still passable from the constructor if needed.

---

8. Known limitations / follow-ups

1. **Custom block paths** — auto-detection covers Qwen2.5-VL and MedGemma. Other backbones need either a path patch in `_get_vit_blocks` or a thin subclass.
2. **`infer_vpt.py` LoRA loading** uses `model.load_adapter(...)` which will fail if no PEFT adapter dir is present at the checkpoint path. For non-LoRA ablations (A1–A3), pass `--no-add_lora`.
3. **LAVE cost / latency** — GPT-4.1-mini at 1,400 test pairs × ~10 KB prompts is on the order of pennies, but rate-limit handling is best-effort (failed batches default to `0.0`).
4. **Validation split** is read from `data/chest_x-ray/val/...` if present. Producing it (stratified 10% from train) is not yet automated; protocol §9 describes the policy.
5. **Gradient checkpointing + hooks** — works, but the pre-hook closure captures `_current_alpha` from `self`; setting `alpha` once before the model call and not mutating it during backward is what makes this safe. Don't move `set_routing_weight` inside checkpointed regions.

---

9. Running on Lightning.ai (cloud GPU)

Lightning.ai gives you a Studio with a persistent root filesystem, optional GPU machines (A100 40 GB / 80 GB / H100), and a Cursor-like web IDE plus terminal. The path below assumes a fresh Studio.

9.1 One-time setup (CPU machine type is fine for setup)

```
# 1. Open a Studio, switch the machine type to CPU for cheap setup, then:
git clone https://github.com/<your-fork>/med-vqa-LoRA-vpt-test.git
cd med-vqa-LoRA-vpt-test

# 2. Install deps. Lightning.ai Studios ship with Python 3.10 and CUDA-ready
        PyTorch.
pip install -r requirements.txt
pip install -r requirements_vpt.txt

# 3. Authenticate Hugging Face (needed for Qwen / MedGemma weights)
huggingface-cli login   # paste a read token from huggingface.co/settings/tokens

# 4. (Optional) Weights & Biases
wandb login

# 5. Pull or place the BanglaMedVQA images
bash download.sh        # or rsync / lightning data upload, depending on your
        setup

# 6. Build the ShareGPT JSONs from CSVs (idempotent)
python prepare_data.py --all
```

9.2 Switch to a GPU machine

In the Studio sidebar: - **Machine** → **Change** → **A100 (80 GB)** for Qwen2.5-VL-7B (recommended). - For Med-Gemma-4B, **A100 (40 GB)** is sufficient. - The persistent disk follows you, so all setup above survives the swap.

Verify GPU:

```
nvidia-smi
python -c "import torch; print(torch.cuda.get_device_name(0),
        torch.cuda.is_available())"
```

9.3 Reproduce the LoRA baseline (B0)

```
# Train
python train_model.py --config configs/qwen2.5-vl-7b.yaml

# Generate predictions on the test set (writes output/chest_x-
        ray_test_predictions.csv)
python train_model.py --config configs/qwen2.5-vl-7b-test.yaml

# Score
python evaluate.py \
  --predictions_csv output/chest_x-ray_test_predictions.csv \
  --output_json results/B0_results.json
```

Verify the per-category Acc / LAVE numbers are within ±1% of paper Table 4 before moving on.

9.4 Train each VPT ablation

Use the YAML configs (CLI flags still override anything inside):

```
# A1 — shared pool, no router, no LoRA
python train_vpt.py --config configs/vpt/A1_shared_no_router.yaml \
                    --gradient_checkpointing

# A2 — shared pool + router, no LoRA
python train_vpt.py --config configs/vpt/A2_shared_with_router.yaml \
                    --gradient_checkpointing

# A3 — split pools + router, no LoRA
python train_vpt.py --config configs/vpt/A3_split_with_router.yaml \
                    --gradient_checkpointing

# A4 — proposed method (split + router + LoRA)
python train_vpt.py --config configs/vpt/A4_split_router_lora.yaml \
                    --gradient_checkpointing

# A5 — oracle router upper bound
python train_vpt.py --config configs/vpt/A5_oracle_router.yaml \
                    --gradient_checkpointing
```

Each run writes: - `output/A?/checkpoint-epoch{1,2,3}/vpt_router.pt` - `output/A?/checkpoint-epoch{1,2,3}/adapter_*.safetensors` (when LoRA is on) - `output/A?/train_loss.csv`

9.5 Inference + scoring per ablation

```
for COND in A1 A2 A3 A4 A5; do
  # A1, A2, A3 do not have LoRA; pass --no-add_lora.
  EXTRA="--add_lora"
  if [[ "$COND" == "A1" || "$COND" == "A2" || "$COND" == "A3" ]]; then
    EXTRA="--no-add_lora"
  fi
  ORACLE=""
  if [[ "$COND" == "A5" ]]; then
    ORACLE="--use_oracle_router"
  fi
  DISABLE=""
  if [[ "$COND" == "A1" ]]; then
    DISABLE="--disable_router"
  fi

  python infer_vpt.py \
    --checkpoint_dir output/${COND}/checkpoint-epoch3 \
    --test_json data/chest_x-ray/test/chest_x-ray_dataset.json \
```

```
    --test_csv  data/chest_x-ray/test/chest_x-ray.csv \
    --output_csv output/${COND}/chest_x-ray_test_predictions.csv \
    $EXTRA $ORACLE $DISABLE

  python evaluate.py \
    --predictions_csv output/${COND}/chest_x-ray_test_predictions.csv \
    --output_json     results/${COND}_results.json
done
```

For LAVE: `export OPENAI_API_KEY=sk-...` before the loop, otherwise pass `--no_lave` to `evaluate.py`.

9.6 Repeat A4 with Med-Gemma backbone

```
python train_vpt.py \
  --config configs/vpt/A4_split_router_lora.yaml \
  --model_name google/medgemma-4b-it \
  --output_dir output/A4_medgemma \
  --gradient_checkpointing

python infer_vpt.py \
  --checkpoint_dir output/A4_medgemma/checkpoint-epoch3 \
  --model_name google/medgemma-4b-it \
  --test_json data/chest_x-ray/test/chest_x-ray_dataset.json \
  --test_csv  data/chest_x-ray/test/chest_x-ray.csv \
  --output_csv output/A4_medgemma/chest_x-ray_test_predictions.csv

python evaluate.py \
  --predictions_csv output/A4_medgemma/chest_x-ray_test_predictions.csv \
  --output_json     results/A4_medgemma_results.json
```

9.7 Cost / time guidance

Run	Machine	Wall time (rough)
B0 baseline (2 epochs, 4,900 samples)	A100 80 GB	~40 min
Each A1–A5 (3 epochs)	A100 80 GB	~60–80 min
All five ablations + inference + LAVE	A100 80 GB	~7–9 GPU-hours

Stop the GPU instance from the Lightning sidebar (**Stop machine**) the moment a run finishes — Lightning bills by GPU-minute, and the persistent disk costs nothing while idle.

9.8 Troubleshooting on Lightning.ai

- **OOM during A4** → ensure `--gradient_checkpointing` is on; drop `--batch_size` to 2 with `--num_epochs 4` to compensate.

- **`sentence-transformers` slow first run** → it downloads ~470 MB on first import; pre-warm by running `python -c "from sentence_transformers import SentenceTransformer; SentenceTransformer('paraphrase-multilingual-mpnet-base-v2')"` once in the setup phase.
- **HF rate-limited** → set `HF_HUB_ENABLE_HF_TRANSFER=1` and retry; bumping a cached `model_name` once is enough.
- **Inference: LoRA not loading** → confirm `output/A?/checkpoint-epoch3/adapter_config.json` exists; if not, the run died before the first checkpoint and you need to re-train.

---

10. Quick file reference

File	Purpose
`src/vpt/prompt_learner.py`	Two prompt pools + Deep VPT hook injection / strip
`src/vpt/category_router.py`	Frozen multilingual encoder + 2-layer MLP → α
`src/vpt/vpt_model.py`	Combined VLM + VPT + router + LoRA
`train_vpt.py`	Two-phase training loop, dataset, loss, checkpointing
`infer_vpt.py`	Reload checkpoint and emit predictions CSV
`evaluate.py`	Per-category Accuracy / BERTScore / LAVE
`train_model.py`	Original LoRA-only baseline (B0) and its inference path
`prepare_data.py`	CSV → ShareGPT JSON conversion
`configs/vpt/A?_*.yaml`	One YAML per ablation; loaded via `train_vpt.py --config`